

AirGravQC

An open-source Python package to QC airborne gravimetry & gradiometry

MARK DRANSFIELD CONSULTING

AirGravQC

... an open-source Python package for QC of airborne gravimetry & gradiometry

- What is it?
- Why do this?
- Who looks after it?
- What is the current status?
- Can we make changes?
- What is inside, code, tutorial(s), docs?
- How do you use it?

This whole Python package, open-source thing is new to me. Comments, criticisms, suggestions, feedback, and help are all wished for.

What is it?

... more on this later

- An open source Python package
- Residing on GA's public Github account
- Includes
 - source code
 - documentation
 - tutorial(s) and examples
 - tests and test data

Why do this?

- To support the Gravity Deed
- To support good standards in airborne gravity
- To fill a clear gap in resources for practicing geophysicists
- To enable a broadening of skills amongst practicing geophysicists
- To provide a starting point from which we can improve

Who looks after it?

- Open source code should be improved
- It always needs maintenance
- If enough people find it useful, some will look after it. For now ...
- I'll be looking after it for some time. I hope others will get involved.

What is the current status?

AirGravQC

- It works and is in constant use.
- There is a tutorial to guide new users through the use of the package.
- The documentation is kind of ok.
- There is hardly any test code.
- Only the developer has ever used it.
- There are stylistic inconsistencies in report and plot outputs.
- There are lots of minor things that could be done better.
- It is not yet released / available.

Can we make changes?

- Right now - ask me to make changes.
- Once it is in the public domain - go for it!
- The usual source control in Github will apply.
- Hopefully, GA will have someone to provide oversight.
- Hopefully, people will make changes, and there will be discussions, and the package will improve.

What is inside?

- ... see the (HTML) documentation ...

How do you use it?

- ... see the tutorial Jupyter-lab notebook ...

lab - JupyterLabconda: Whizz

FileEditViewRunKernelTabsSettingsHelp

TEST_AIRGRAVQC.IPYNB

<>M≡

Testing AirGravQC - Example Data

Preliminary Notes

Introduction...

Import and Summarise...

Reports and formats...

Navigation and positioning...

Gravimeter...

AGG Gradiometer...

FTG Gradiometer

Statistics and Grids...

Test_AirGravQC.ipynb

Python 3 (ipykernel)

Testing AirGravQC - Example Data

Mark Dransfield 10 April 2023

Preliminary Notes

+ 4 cells hidden

Introduction

+ 9 cells hidden

Import and Summarise

Import required modules, and set filenames for data and plan.

First import external packages. Some of these are probably not needed here.

[1]:import sysimport importlib as im# %matplotlib widgetfrom pathlib import Path

Set the paths for the AirGravQC software and import. (One day this will require fewer steps.)

[3]:# Choose `local\_docs` to suit you.local_docs = "/Users/markdransfield/"# local_docs = "/Users/markdransfield/Library/Mobile Documents/com~apple~CloudDocs/"src_path = local_docs + "Documents/GitHub/AirGravQC/AirGravQC"sys.path.insert(0, src_path)import pointfiles as mhdimport qualityAnalysis as qcimport whizzPlot as wpimport gridfiles as erm

Set the paths for the data. Usually the data would be from only one project but here we are including a variety of exemplars.

Modify to suit your data.

[5]:data_root = local_docs + "Documents/GitHub/AirGravQC/examples/SourceData/"coodaXYZ_file = Path(data_root + r'Coodabeen.xyz')coodaHDF_file = coodaXYZ_file.with_suffix('.hdf5')quixoteXYZ_file = Path(data_root + r'Quixote.xyz')quixoteHDF_file = quixoteXYZ_file.with_suffix('.hdf5')quixoteTestXYZ_file = Path(data_root + r'QuixoteTest.xyz')quixoteTestHDF_file = quixoteTestXYZ_file.with_suffix('.hdf5')quixoteRepXYZ_file = Path(data_root + r'QuixoteRepeats.xyz')quixoteRepHDF_file = quixoteRepXYZ_file.with_suffix('.hdf5')stumpXYZ_file = Path(data_root + r'Blackstump.xyz')

Simple0\$1Python 3 (ipykernel) | IdleMode: CommandLn 1, Col 1Test_AirGravQC.ipynb

//Documents/GitHub/AirGravQC/docs/build/html/index.html

Profiling · The Julia...Crash course · Dime...Variables · NCDatas...Rasters.jl · Rasters.jlRegionTrees.jl/hype...Airborne Gravity Qu...

AirGravQC

Navigation

AboutGetting StartedgeoWhizzCookbooksMethodsAirborne Gravity DeedAPI

Quick search

Go

Airborne Gravity Quality Control

AirGravQC is a Python library for quality assurance and control of airborne gravity data. It covers both gravimetry and gradiometry from a range of sensor models and was designed for use in QC of data collected under contracts based on the Geoscience Australia Airborne Gravity Deed [\[1\]](#) , although it is useful for the QC of most airborne gravity projects.

Note:

This project is under active development.

Contents (see also [API](#))

About

Citing

Contributing

Licence

The Airborne Gravity Deed

Getting Started

Installation

Session Setup

geoWhizz

Introduction

The root group

The Coordinate Frame

The Lines Group

The Dataset Group

Cookbooks

Methods

Prepare data

Convert XYZ to geoWhizz

Project Meta-data

Line numbering system

Channel meta-data

Report Contents

Report Flights

Report Sampling

The Plan

Lines Map

Navigation and Positioning

GNSS Satellites

Horizontal Position

Drape Following

Segment Length

Minimum Overlap

Minimum Clearance

Ground Speed

Gravimetry

Atmospheric Correction

Latitude Correction

Free-air Correction

Eotvos Correction

AGG Gradiometry

AGG Difference Noise

FTG Gradiometry

In Line Sum (ILS)

Aeromagnetics

Break from slides to review:

- **the HTML documentation and**
- **the Jupyter notebook tutorial.**

Questions?